

UCLouvain

LINFO1121 2026-2027

Algorithmique et Structure de données

Restructuration 3: Search Trees

TA LLN: Harold Kiossou, Alice Burlats, Achille Morenville (Competition) + Tutors
TA Charleroi: Roger Kameugne

Prof pierre.schaus@uclouvain.be

Binary Search: True or False ?

- In the best case, is the number of key comparisons for a binary search of a particular key in a sorted array of N distinct keys ~log(N)

True or false ?

- In the best case, is the number of key comparisons for a binary search of a particular key in a sorted array of N distinct keys ~log(N)

- False: That corresponds to the worst case; in the best case, you hit the key directly on the first try, so it's O(1).

Reminder: Tree traversals

```
preOrder(BinaryTree A) :
    visit (A)
    If nonEmpty (left(A))
        preOrder(left(A))
    If nonEmpty (right(A))
        preOrder(right(A))
```

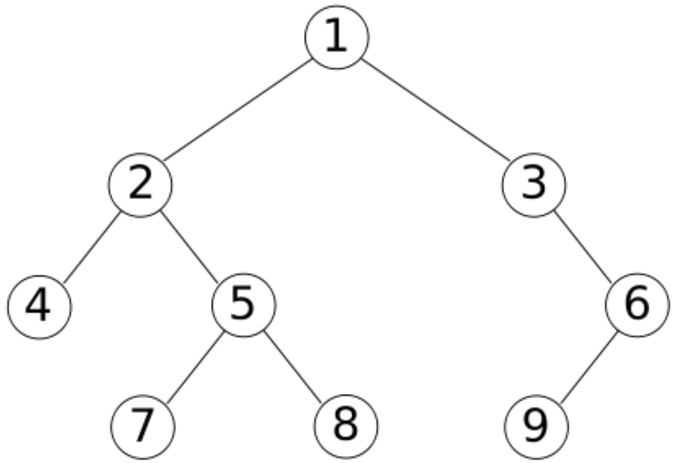
1, 2, 4, 5, 7, 8, 3, 6, 9

```
postOrder(BinaryTree A) :
    If nonEmpty(left(A))
        postOrder(left(A))
    If nonEmpty(right(A))
        postOrder(right(A))
    visit(A)
```

4, 7, 8, 5, 2, 9, 6, 3, 1

```
infixOrder(BinaryTree A) :
    If nonEmpty(left(A))
        infixOrder(left(A))
    visit(A)
    If nonEmpty(right(A))
        visitInfix(right(A))
```

4, 2, 7, 5, 8, 1, 3, 9, 6



!!Not a BST!!

BST True or False?

- Given an inorder traversal of a BST containing N distinct keys:

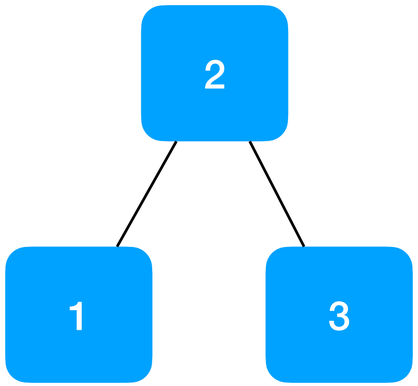
- Is it possible to reconstruct the shape of the BST based on the traversal result?

- If yes, write the pseudocode for an algorithm to do so; if no, provide a counterexample justifying your answer.

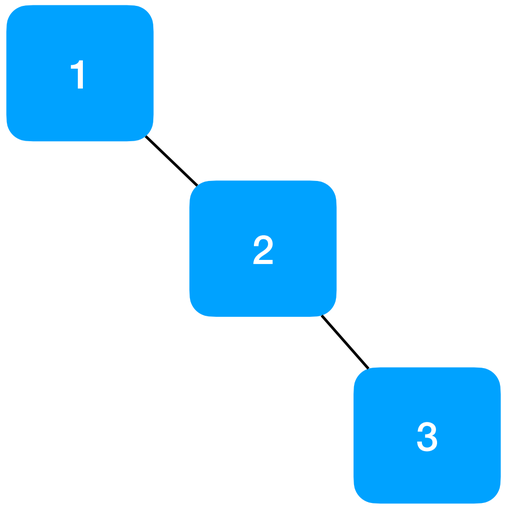
Counter Example

- infix(T1) = infix(T2) = 1-2-3 et T1 != T2

- T1



- T2



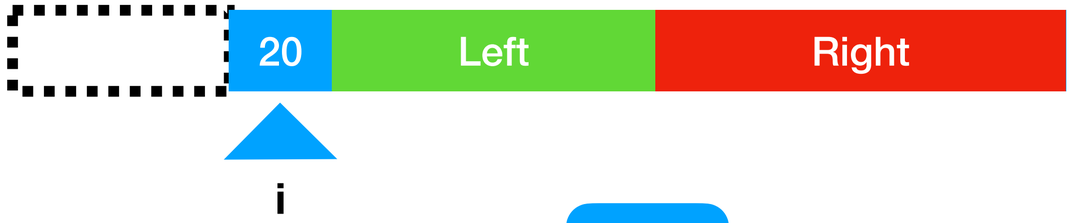
True or False ?

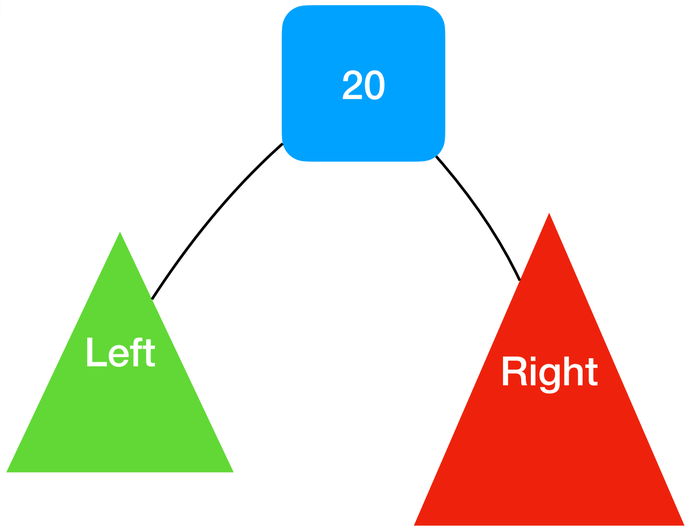
- Given a preorder traversal of a BST containing N distinct keys:

- Is it possible to reconstruct the shape of the BST based on the traversal result?

- If yes, write the pseudocode for an algorithm to do so; if no, provide a counterexample justifying your answer.

Algorithm Idea





Node[Integer]

Node left
Node right
Integer key
Int size

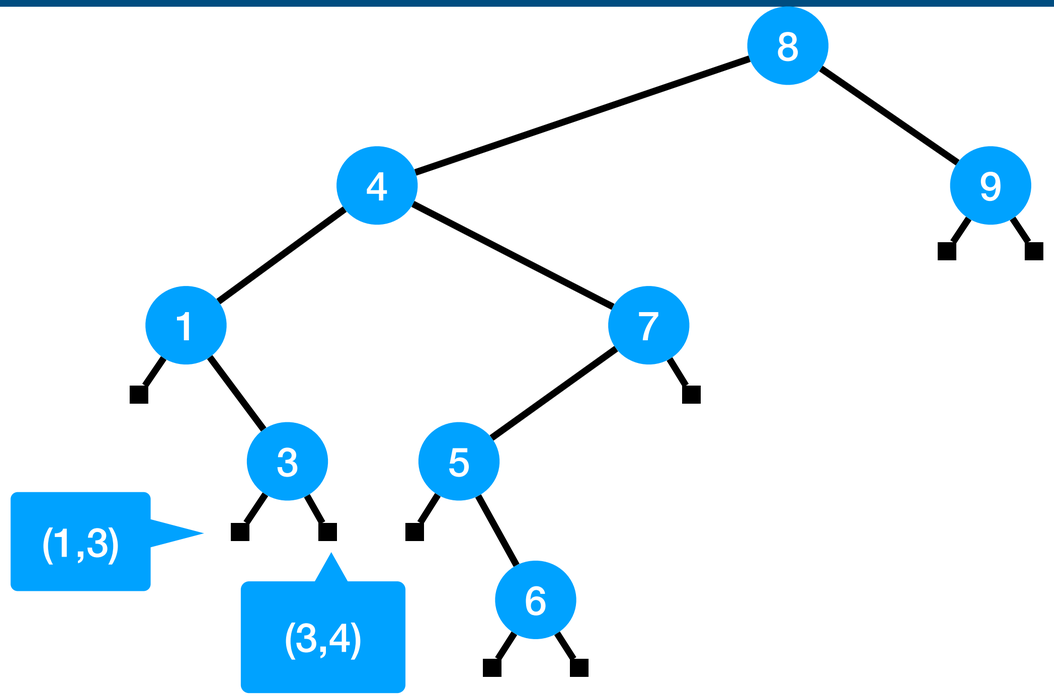
```
private Node readBSTFromPreorder(int[] input, int i, ...) {
    if (i >= input.length || ...) {
        return null;
    }
    Node left = readBSTFromPreorder(input, i + 1, ...);
    int leftSize = (left == null) ? 0 : left.size;
    Node right = readBSTFromPreorder(input, i + leftSize +
1, ...);
    return new Node(input[i], left, right);
}
```

What is this a useful result ?

- What about the leaves ?

8,4,1,3,7,5,6,9





- When $i = 3$, we recursively trigger from $i = 4$ (key $= 7$) the reading of the left tree:

- Why can't 7 be a right child of 3, given that the prefix 8, 4, 1, 3 has already been read?

  - Because we expect a key between 1 and 3 on the left, and between 3 and 4 on the right.

Algo: Recursive Procedure

```
public static Node readBSTFromPreorder(int [] input) {
   return readBSTFromPreorder(input,0,Integer.MIN_VALUE, Integer.MAX_VALUE);
}
```

Reads the longest possible BST rooted at node input[i] with all its keys between min and max.
@param input the keys of a BST ordered by preorder traversal
@param i a position in input
@param min the minimum key value
@param max the maximum key value

```
private Node preorderRead(int [] preOrderInput, int i, int min, int max) {
    if (i >= preOrderInput.length) return null;
    if (preOrderInput[i] > max || preOrderInput[i] < min) return null;
    Node left = preorderRead(preOrderInput, i + 1, min, preOrderInput[i]);
    int leftSize = left != null ? left.size : 0;
    Node right = preorderRead(preOrderInput, i + 1 + leftSize, preOrderInput[i], max);
    return new Node(left, right, preOrderInput[i]);
}
```

What about this solution ?

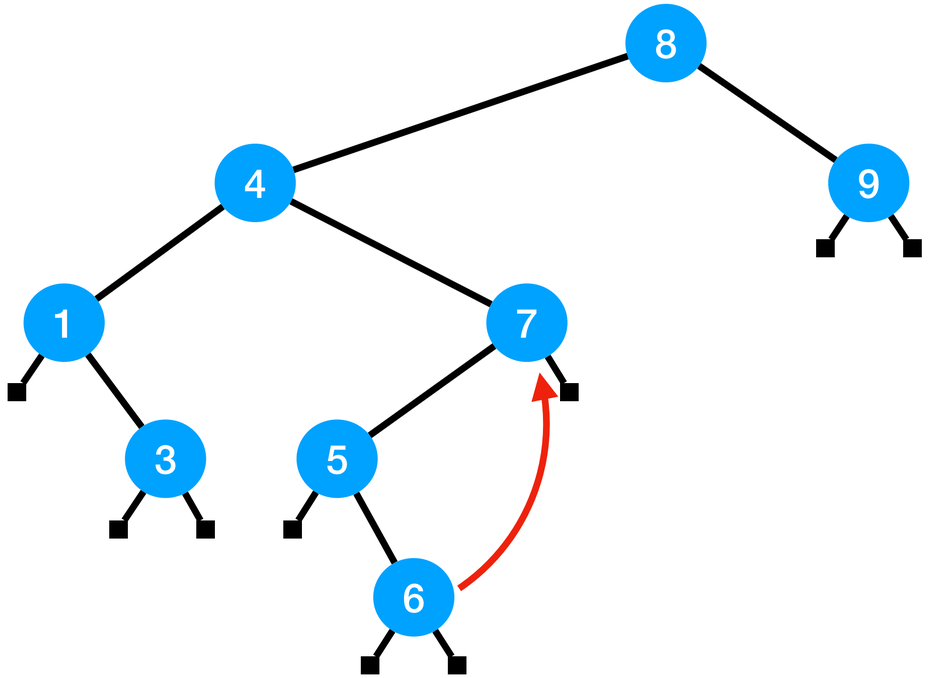
```
public PreorderToBST(int [] preOrderInput) {
    Node head = new Node(null,null,preOrderInput[0]);
    for (int e: preOrderInput) {
        put(head, e);
    }
    return head;
}
```

What is the time complexity for this input:

[1,2,3,4,…,n] ?

True or False?

- Given a BST of N distinct keys and a given key x, is it possible to find the smallest key larger than X in logarithmic time complexity in the worst case?



True or False?

- Given a BST of N distinct keys and a given key x, is it possible to find the smallest key larger than X in logarithmic time complexity in the worst case?

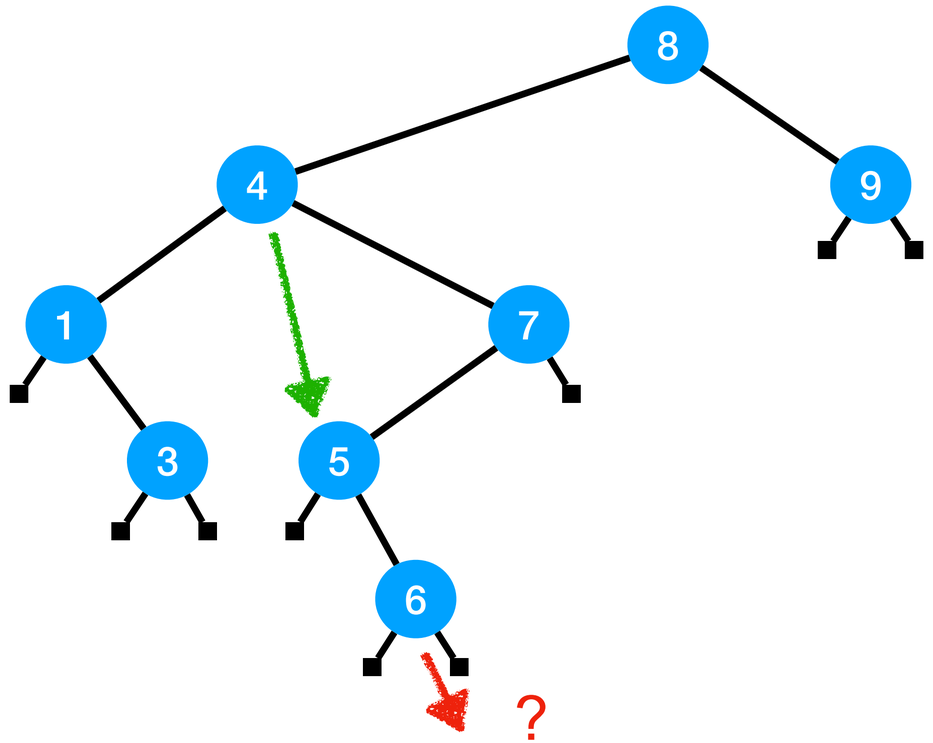
- FALSE. Height of a BST of N keys randomly inserted is expected to be logarithmic but nothing is mentioned here about randomness, nor that the BST is balanced so in the worst case h can be in O(n).

True or False ?

- Let x be a node/key in a BST. The successoral of x (node containing next key in the order) is the left most tree in the right tree of x ?

True or False?

- Let x be a node/key in a BST. The successoral of x (node containing next key in the order) is the left most tree in the right tree of x ? False! Successor of 6.



True or False?

- The max height of a 2-3 tree with N keys is ~$\log_3 N$?

True or False?

- The max height of a 2-3 tree with N keys is ~$\log_3 N$?

- The max height of a 2-3 tree is within
  - $\lfloor \log_3(N) \rfloor \leq h(2-3\text{tree}) \leq \lfloor \log_2(N) \rfloor$

All 3 nodes

All 2 nodes

  - Thus False, as ~$\log_3 N$ is the min height of a 2-3 tree. The order of growth (~) takes the multiplicative constants into account, so ~$\log_3 N \neq$ ~$\log_2 N$
  - $\log_2 N = \log_3(N)/\log_3(2)$ (to move from one to the next, you introduce a constant).
  - The max height of a 2-3 tree with N keys is $\Theta(\log_2(N))$

True or False

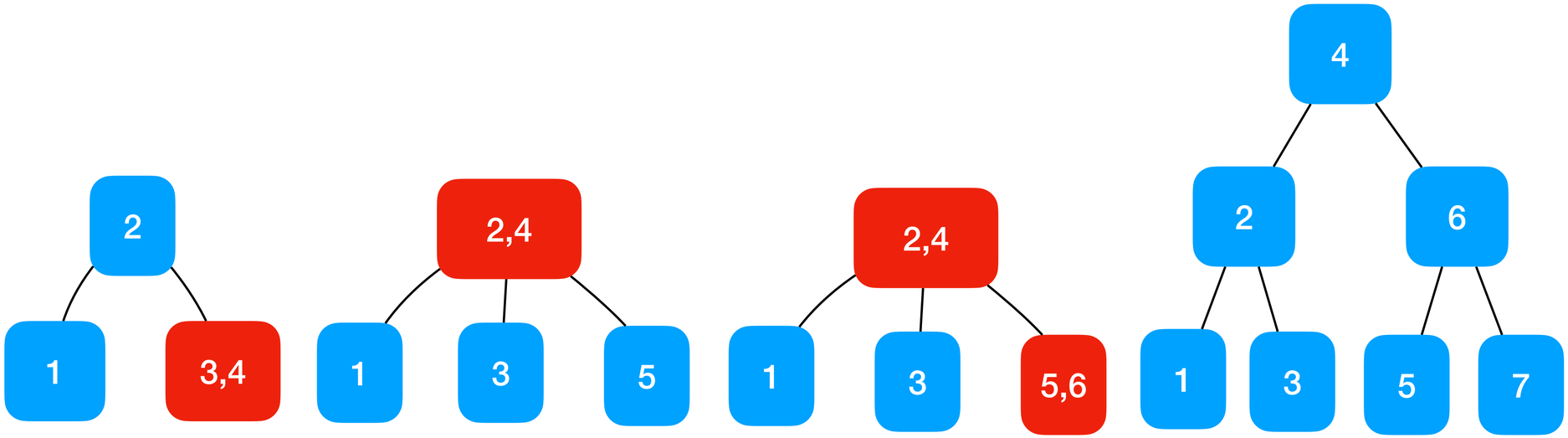
- For the insertion of N keys in increasing order into an initially empty red-black BST, the number of color flips for the last insertion is at most 3. The number of color flips is defined as the absolute difference between the number of red nodes after the insertion and the number of red nodes before the insertion.



- Let's translate the question: *For the insertion of N keys in increasing order into an initially empty 2-3 tree, during the last insertion, at most three 3-nodes were eliminated*

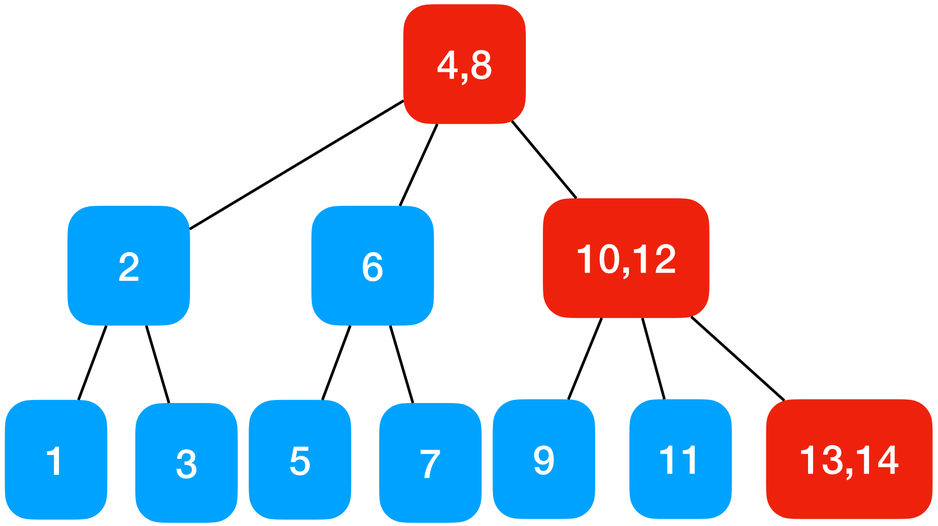
Let’s check

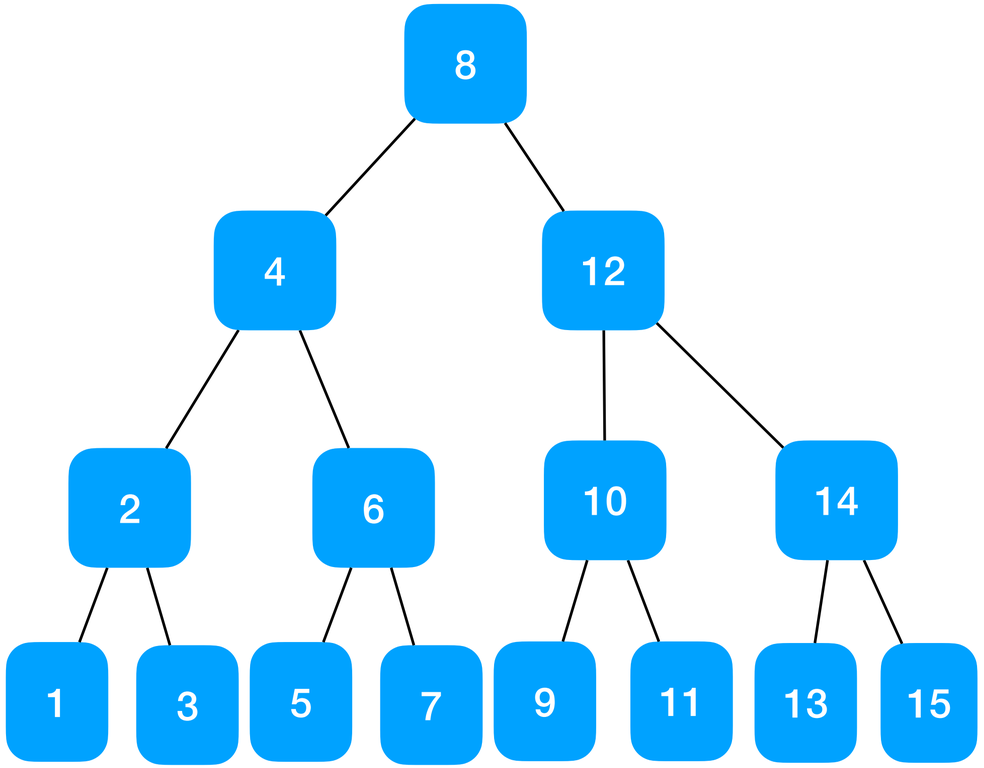
- We insert 1,2,3,4,5,…



- Let’s try to find a counter example
- We insert 1,2,3,4,5,6,**7 this last insert supresses 2x (3-nodes).**
- **For the insertion of 15, the number of changes will be 3 because all the right most branch will contain 3-nodes that will burst along it to the root**

- After this 1,2,3,4,5,6,7,8,9,10,11,12,13,14

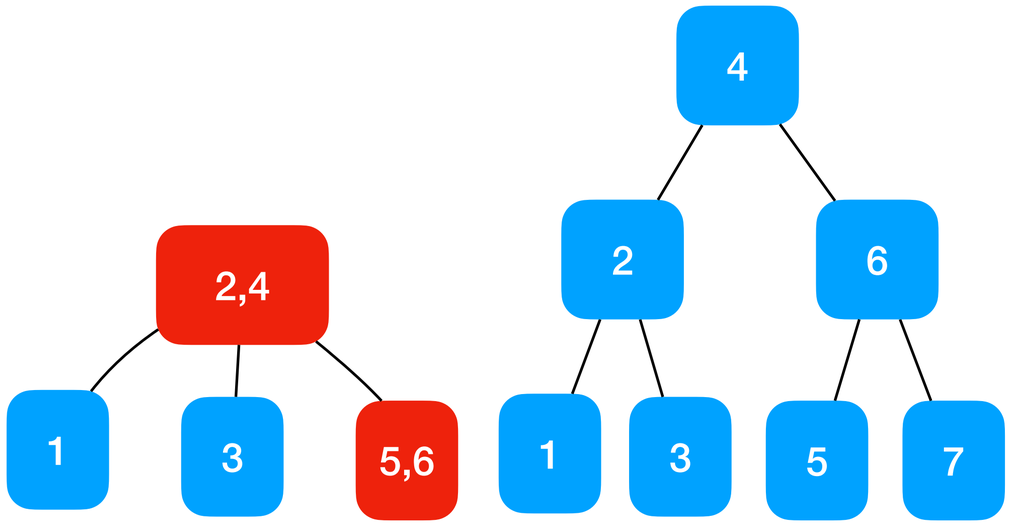




True of False

- A red-black BST obtained after inserting N>1 keys into an initially empty tree has at least one red link.

- Translate: There is at lest one 3-node in a 2-3 tree ?



True or False

- In a red-black BST with N nodes, the black height (i.e., the number of black links on every path from the root to a null link) is at most log N

- Translate: the height of a 2-3 tree is at most log N.
  - **True** since $\lfloor \log_3(N) \rfloor \leq h(2-3\text{tree}) \leq \lfloor \log_2(N) \rfloor$

Sorting using a BST 🤔

- Design a sorting algorithme that uses BST.

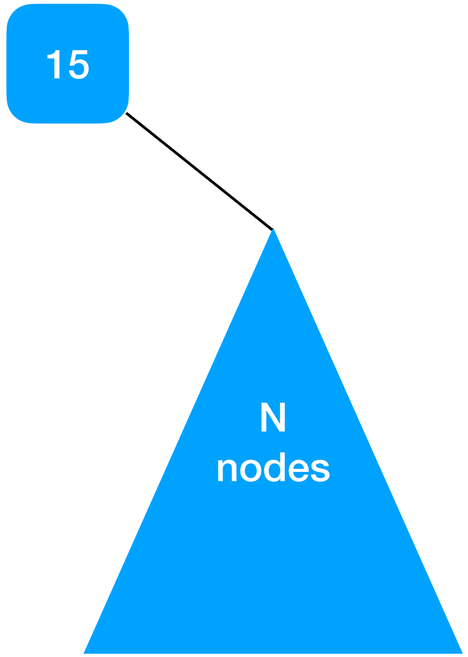
- What would be the time-complexity of this algorithm?

Sorting with a Red-Black Tree ? 🤔

- What if we use a red-black BST ?
  - BST: $O(n^2)$ for the construction BST as insertion is performed in $O(n)$, the $O(n)$ for the infix traversal => $O(n^2)$ overall.
  - Red-black: $O(n \log n)$ for the construction as one insertion takes $O(\log n)$. Then $O(n)$ for the infix traversal => $O(n \log n)$ overall.

Time complexity for supressing key 15 ?

- Using the reference book implementation

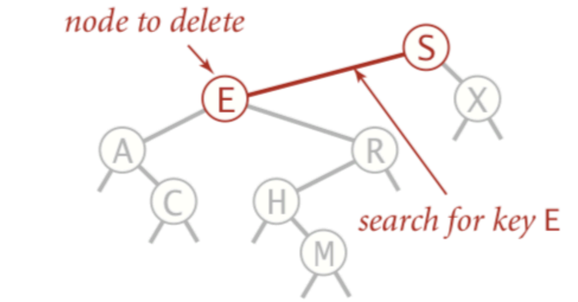


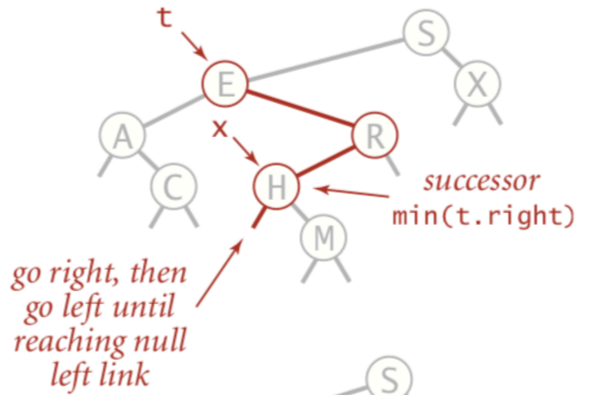
Reminder delete

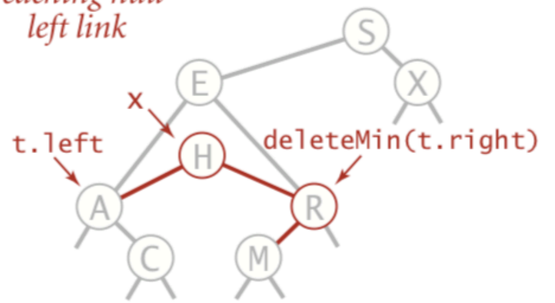
```
public void delete(Key key) {
    root = delete(root, key);
}

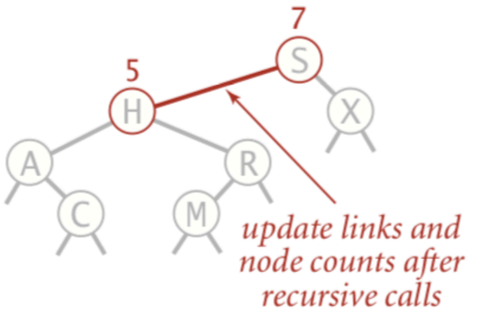
private Node delete(Node x, Key key) {
    if (x == null) return null;
    int cmp = key.compareTo(x.key);
    if (cmp < 0) x.left = delete(x.left, key);
    else if (cmp > 0) x.right = delete(x.right, key);
    else {
        if (x.right == null) return x.left;
        if (x.left == null) return x.right;
        Node t = x;
        x = min(t.right);  // See page 407.
        x.right = deleteMin(t.right);
        x.left = t.left;
    }
    x.N = size(x.left) + size(x.right) + 1;
    return x;
}
```

O(1)







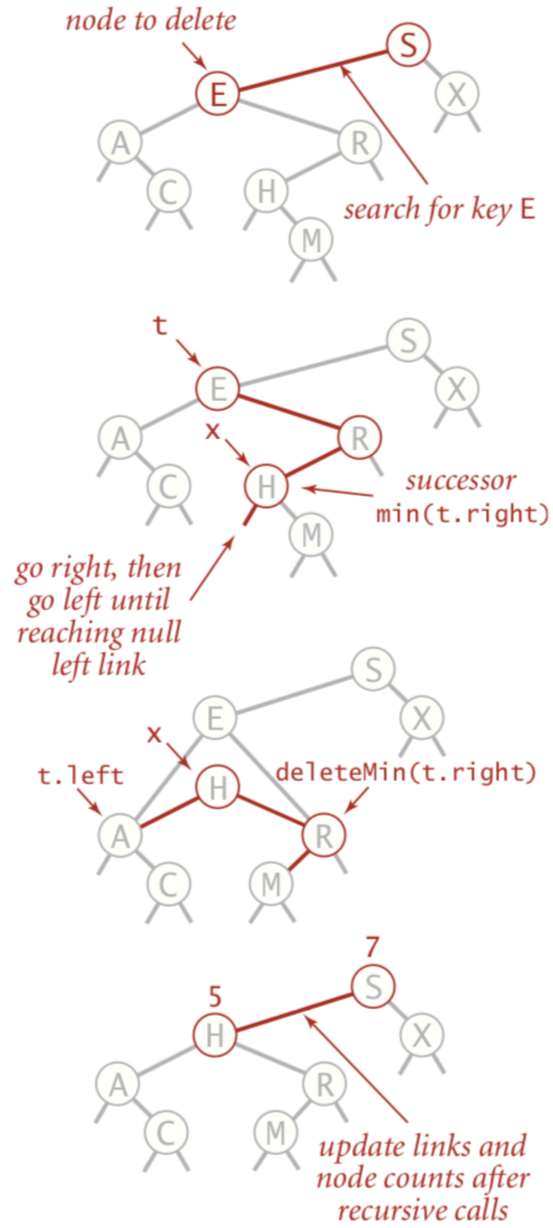


Rappel sur le delete

```
public void delete(Key key) {
    root = delete(root, key);
}

private Node delete(Node x, Key key) {
    if (x == null) return null;
    int cmp = key.compareTo(x.key);
    if (cmp < 0) x.left = delete(x.left, key);
    else if (cmp > 0) x.right = delete(x.right, key);
    else {
        if (x.right == null) return x.left;
        if (x.left == null) return x.right;
        Node t = x;
        x = min(t.right);  // See page 4
        x.right = deleteMin(t.right);
        x.left = t.left;
    }
    x.N = size(x.left) + size(x.right) + 1;
    return x;
}
```

Successor

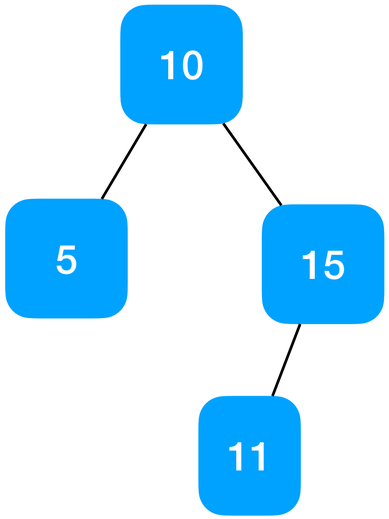


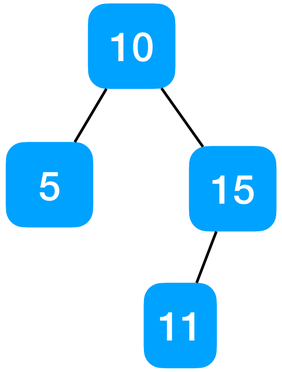
True or false?

- Is the delete operation in a BST **commutative**? That is, does deleting x and then immediately y from a BST (as implemented in the book) leave the tree in the same state as deleting y first and then x?"

- Give a counter example or argue why.

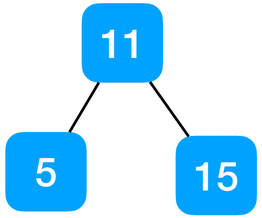
- To help you, consider the following BST and the deletion of 5 and 10.

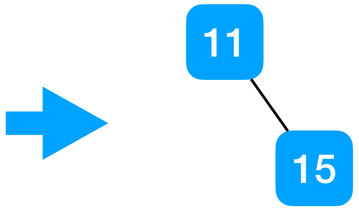




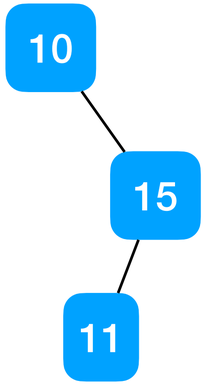

```
public void delete(Key key) {
    root = delete(root, key);
}
private Node delete(Node x, Key key) {
    if (x == null) return null;
    int cmp = key.compareTo(x.key);
    if (cmp < 0) x.left = delete(x.left, key);
    else if (cmp > 0) x.right = delete(x.right, key);
    else {
        if (x.right == null) return x.left;
        if (x.left == null) return x.right;
        Node t = x;
        x = min(t.right);  // See page 407.
        x.right = deleteMin(t.right);
        x.left = t.left;
    }
    x.N = size(x.left) + size(x.right) + 1;
    return x;
}
```

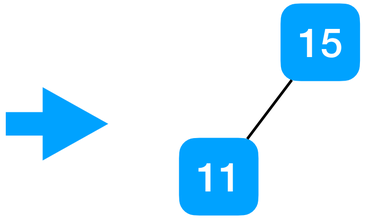
- Delete 10 then 5





- Delete 5 then 10



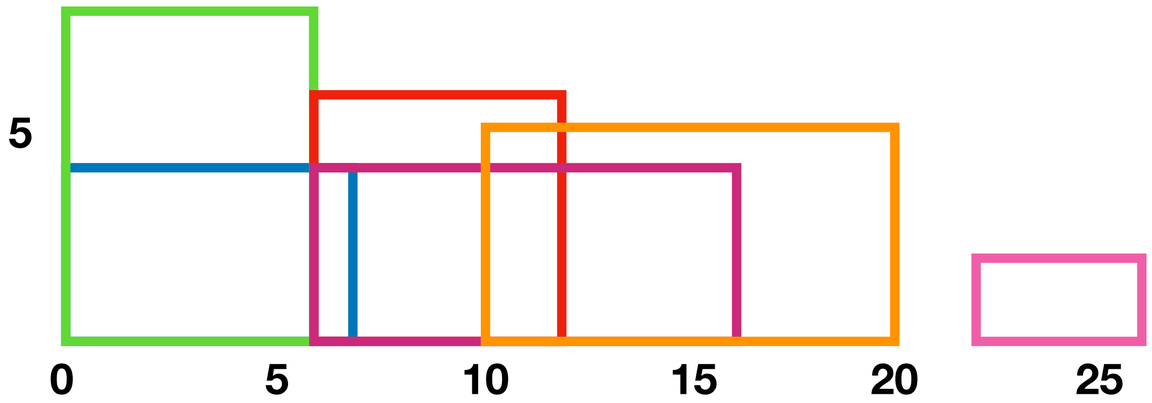


Skyline

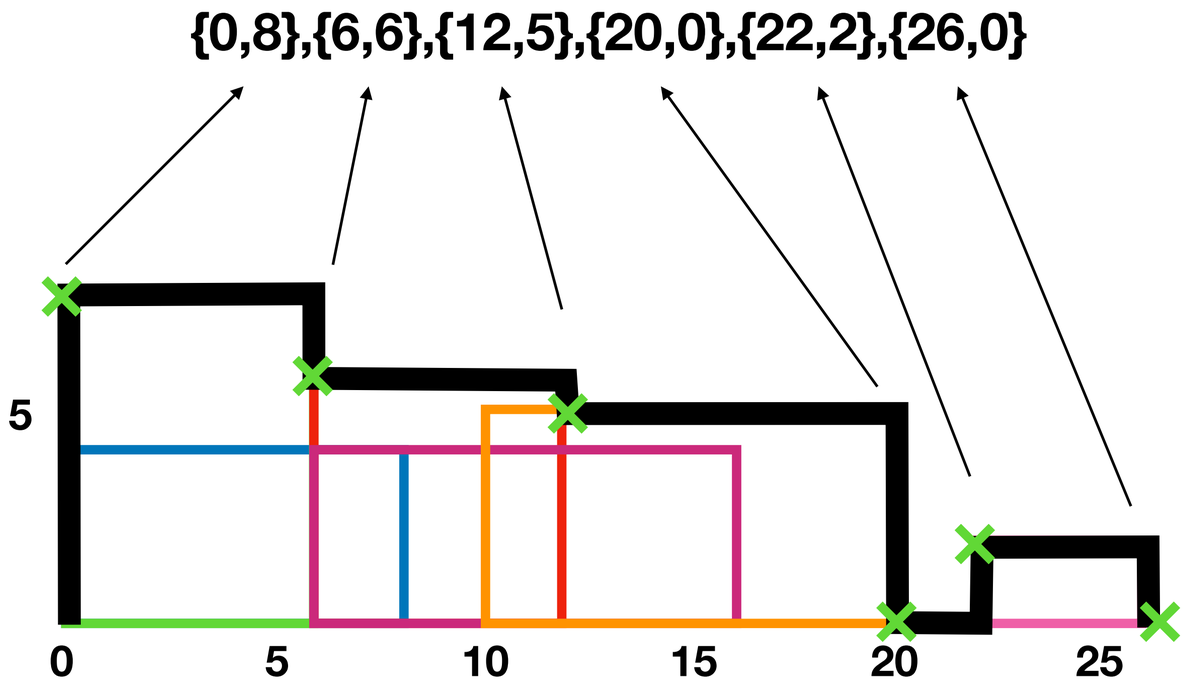


Skyline: Input

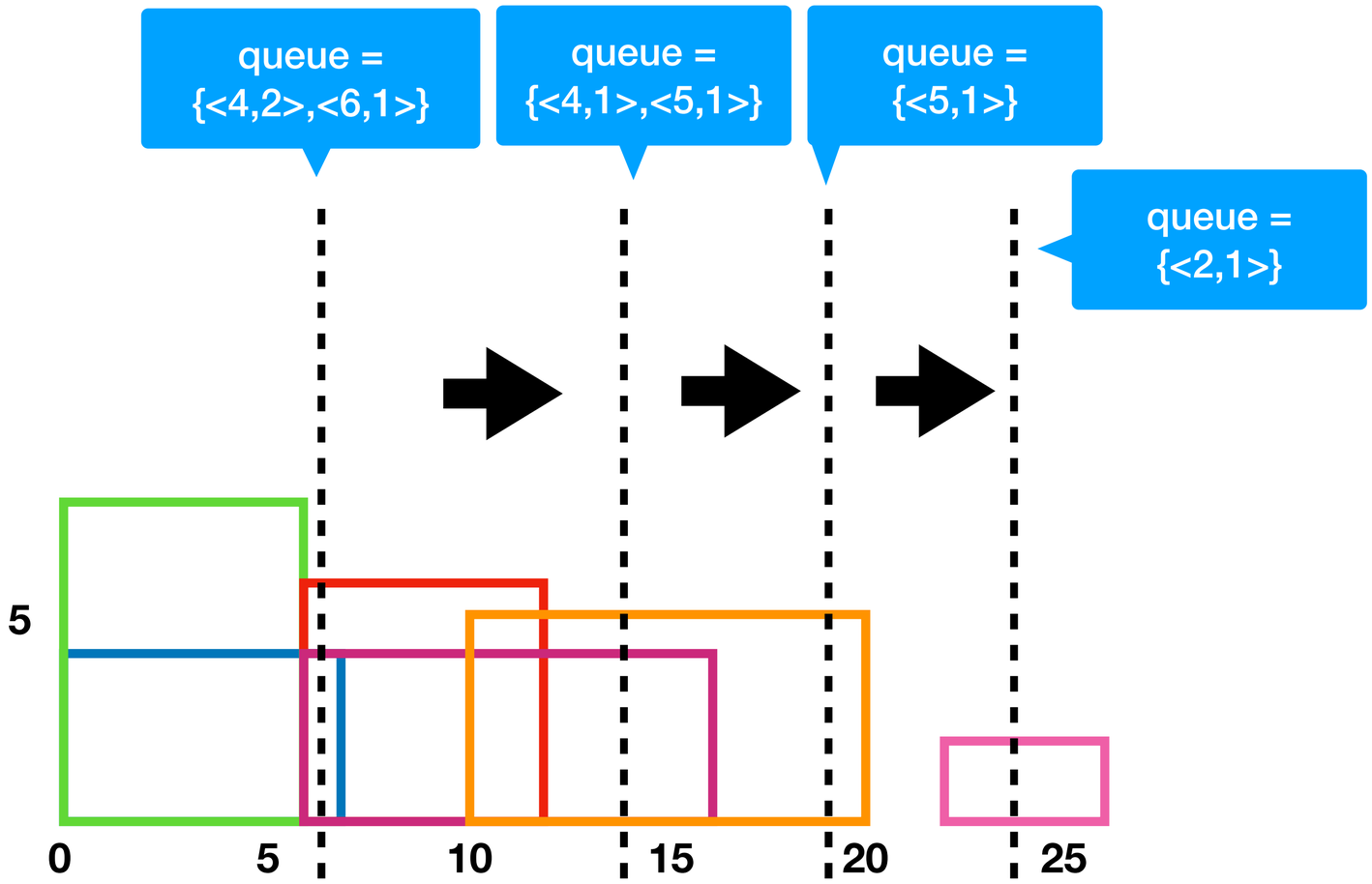
{0,4,7},{0,8,6},{6,6,12},{6,4,16},{10,5,20},{22,2,26}



Sklyline: Output



Sweep Line Algorithm



Observation: the state of the queue can only change at the beginning or at the end of a rectangle

{0,4,7},{0,8,6},{6,6,12},{6,4,16},{10,5,20},{22,2,26}

Sweep Line for Skyline

1. Sort the points (x,h,beginning/end) of the beginning and the end of each building and process each (2n points)

2. Whenever a new point is met with height h:

    1. Beginning: add in the queue an new entry with key=h as corresponding height, or increment the counter if key already present

    2. End: decrement the counter associated with the key=h (already present)

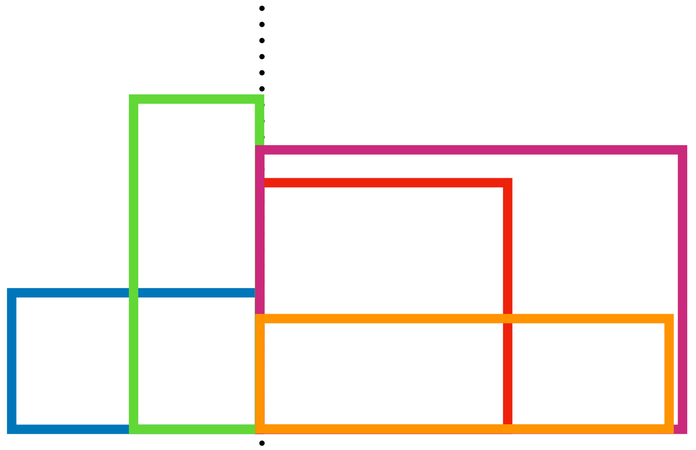
3. A new point of the skyline is created if a new height is detected (when you have processed all the events at this time)

Building Point

```
public static class BuildingPoint implements Comparable<BuildingPoint> {
    int x;
    boolean isStart;
    int height;

    public BuildingPoint(int x, boolean isStart, int height) {
        this.x = x;
        this.isStart = isStart;
        this.height = height;
    }

    @Override
    public int compareTo(BuildingPoint other) {
        if (this.x != other.x) {
            return this.x - other.x;
        } else {
            // always put start points before end points
            // if both are start points, order by decreasing height
            // if both are end points, order by increasing height
            return (this.isStart ? -this.height : this.height) - (other.isStart ? -other.height : other.height);
        }
    }
}
```



Full Algo

```
public static List<int[]> getSkyline(int[][] buildings) {
    // Create a list of points.
    BuildingPoint[] points = new BuildingPoint[buildings.length * 2];
    // Fill in the points
    Arrays.sort(points); // Sort the points.
    // Use a tree map to represent the active buildings.
    TreeMap<Integer, Integer> queue = new TreeMap<>();
    queue.put(0, 1); // Add a ground level (height 0).
    int prevMaxHeight = 0;
    List<int[]> result = new ArrayList<>();
    for (BuildingPoint point : points) {
        if (point.isStart) {
            // If it's a start point, add the height to the map, or increment the existing height's count.
            queue.compute(point.height, (key, value) -> {
                if (value != null) return value + 1;
                return 1;
            });
        } else {
            // If it's an end point, decrement or remove the height from the map.
            queue.compute(point.height, (key, value) -> {
                if (value == 1) return null;
                return value - 1;
            });
        }
        // Get current max height after the addition/removal above.
        int currentMaxHeight = queue.lastKey();
        // If the current max height is different from the previous one, we have a critical point.
        if (prevMaxHeight != currentMaxHeight) {
            result.add(new int[]{point.x, currentMaxHeight});
            prevMaxHeight = currentMaxHeight;
        }
    }
    return result;
}
```

```
// Fill in the points
int index = 0;
for (int[] building : buildings) {
  points[index] = new BuildingPoint(building[0], true, building[1]);
  points[index + 1] = new BuildingPoint(building[2], false, building[1]);
  index += 2;
}
```

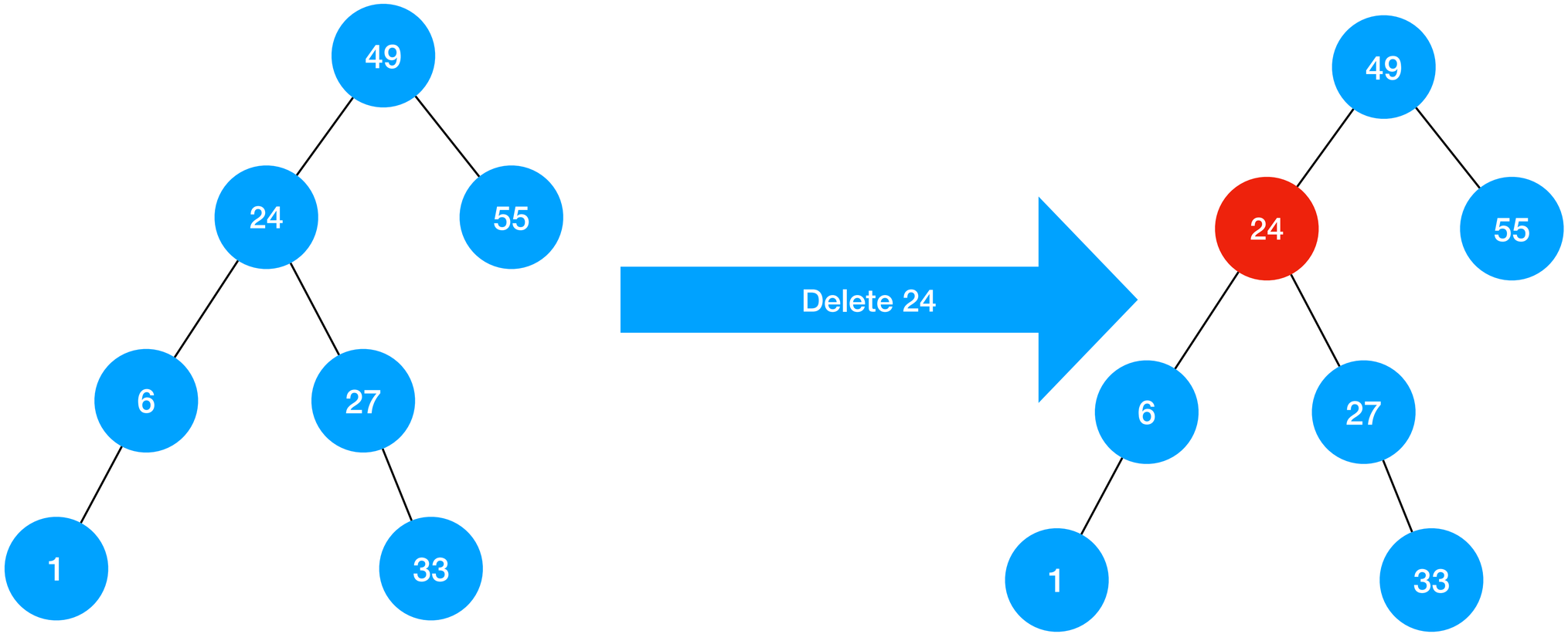
Time Complexity ?

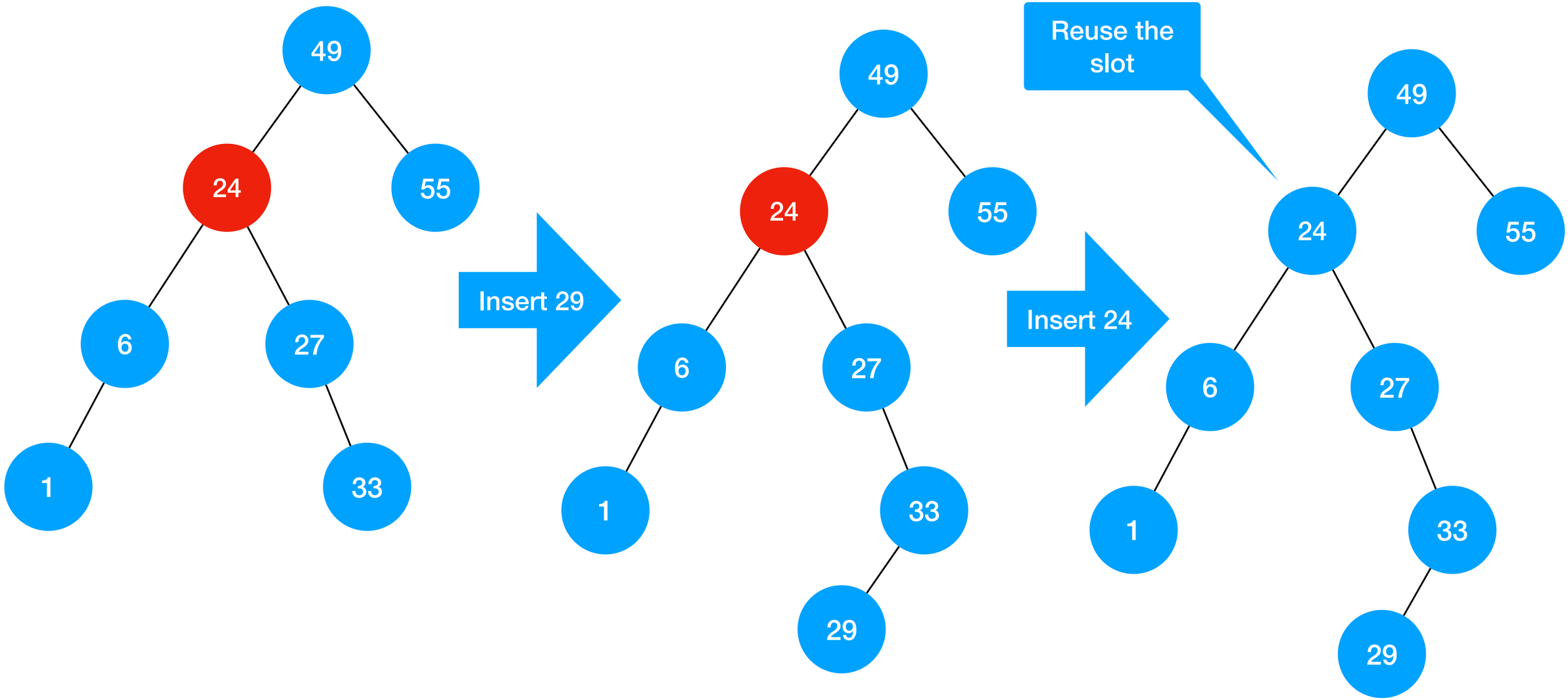
ArrayBST

```
*  The tree below is the result of putting (key,value) pairs (12,A),(15,B),(5,C),(8,d),(1,E) (in this order)
*
*                12(A)
*                / \
*               /   \
*            5(C)  15(B)
*            / \
*         1(E)  8(D)
*
*   The state of internal array list after those put operations is
*    values:        A,  B, C, D, E
*    keys:         12, 15, 5, 8, 1
*    idxLeftNode:   2, -1, 4,-1,-1
*    idxRightNode:  1, -1, 3,-1,-1
```

ArrayBST Delete

- Just add an array-list that represent the fact the the node is not used any more, but keep the BST shape unchanged




49
24
55
6
27
1
33
Insert 29
49
24
55
6
27
1
33
29
Reuse the slot
Insert 24
49
24
55
6
27
1
33
29



UCLouvain

LINFO1121 2026-2027

Algorithmique et Structure de données

Intro 4: Hash Tables

TA LLN: Harold Kiossou, Alice Burlats, Achille Morenville (Competition) + Tutors
TA Charleroi: Roger Kameugne

Prof pierre.schaus@uclouvain.be

Rappel: Table of Symbols

- Abstract Data Type that allows to:
  - **insert** a value attached to key 🗝
  - Given the key 🗝 de **retrieve** the corresponding value

We have seen:

- Balanced Search Trees (red-black trees)

- They allow inserting/deleting a key in $O(\log N)$, and above all, they provide order-based methods on keys:
  - Enumeration in ascending order
  - Finding the minimum, successor, ceil, floor, etc.

- However, this requires keys that are comparable to each other (total order). This is not always possible, and these methods are not always needed.

Hashtable aka Hashmap

- Is a specific dictionary when the only operations we are interested in are:
  - **push(key,value)** and **get(key)**
- It is thus not ordered
- The big difference with red-black is that push/get is performed in O(1) expected rather than O(log(n)) in the worst case. Does it make a big difference?

Let’s try

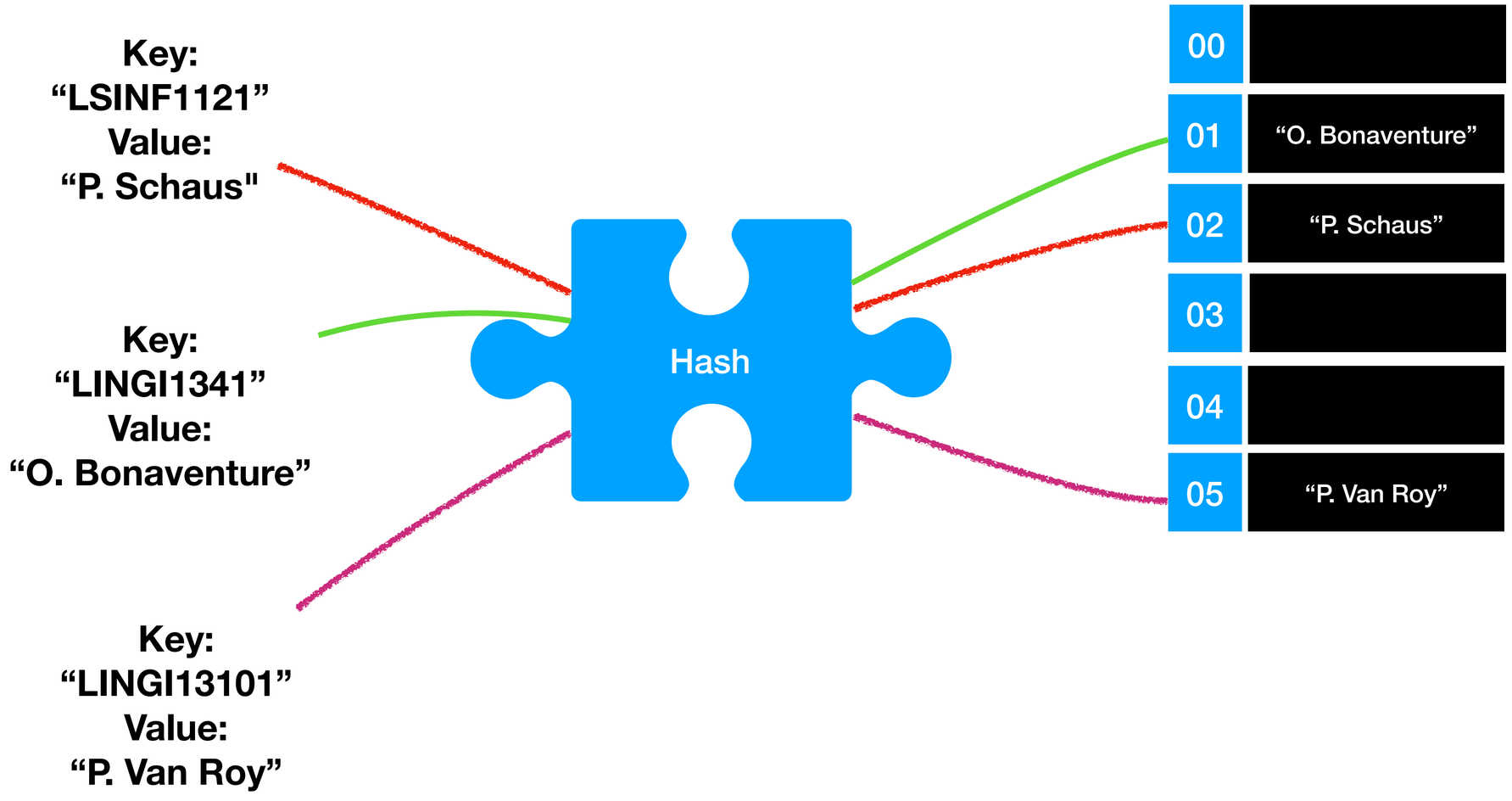
- Speed O(1) vs O(log(N))

```
int size = 10000000;
Random r = new Random();
Integer[] o = new Integer[size];
for (int i=0; i<size; ++i)
    o[i] = Integer.valueOf(r.nextInt());
Map map = new HashMap() ;
map = new TreeMap(); // red-black
long l = System.currentTimeMillis();
for (int i = 0; i < size; i++) {
    map.put(o[i], o[i]);
}
for (int i = size - 1; i > -1; i--) {
    map.remove(o[i]);
}
l = System.currentTimeMillis() - l;
System.out.println("time = " + l);
```

- Results:
  - 4s HashMap
  - 27s TreeMap (red-black in java)

Hash tables rely on Hash functions (that can be user-provided)

- A hash function transforms an object (deterministically) as an integer that will be used as an index in an array.
- The more random the number looks like, the better it is



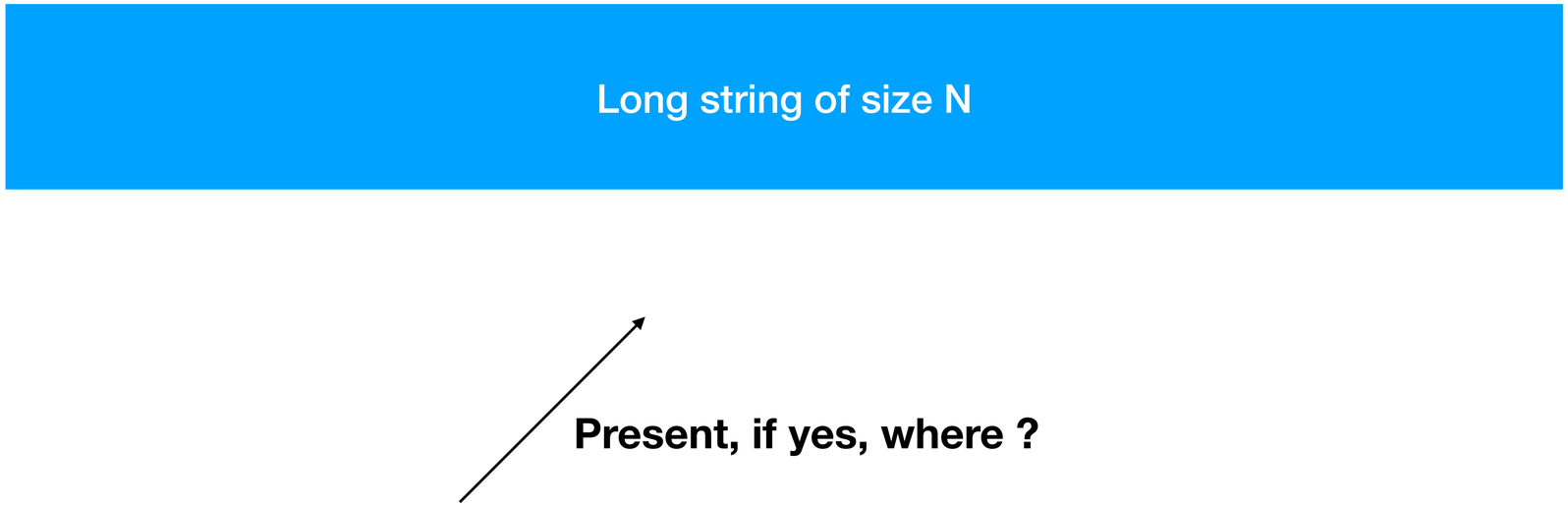
Rabin-Karp algorithm

- A good Hash hash function should behave like a dice

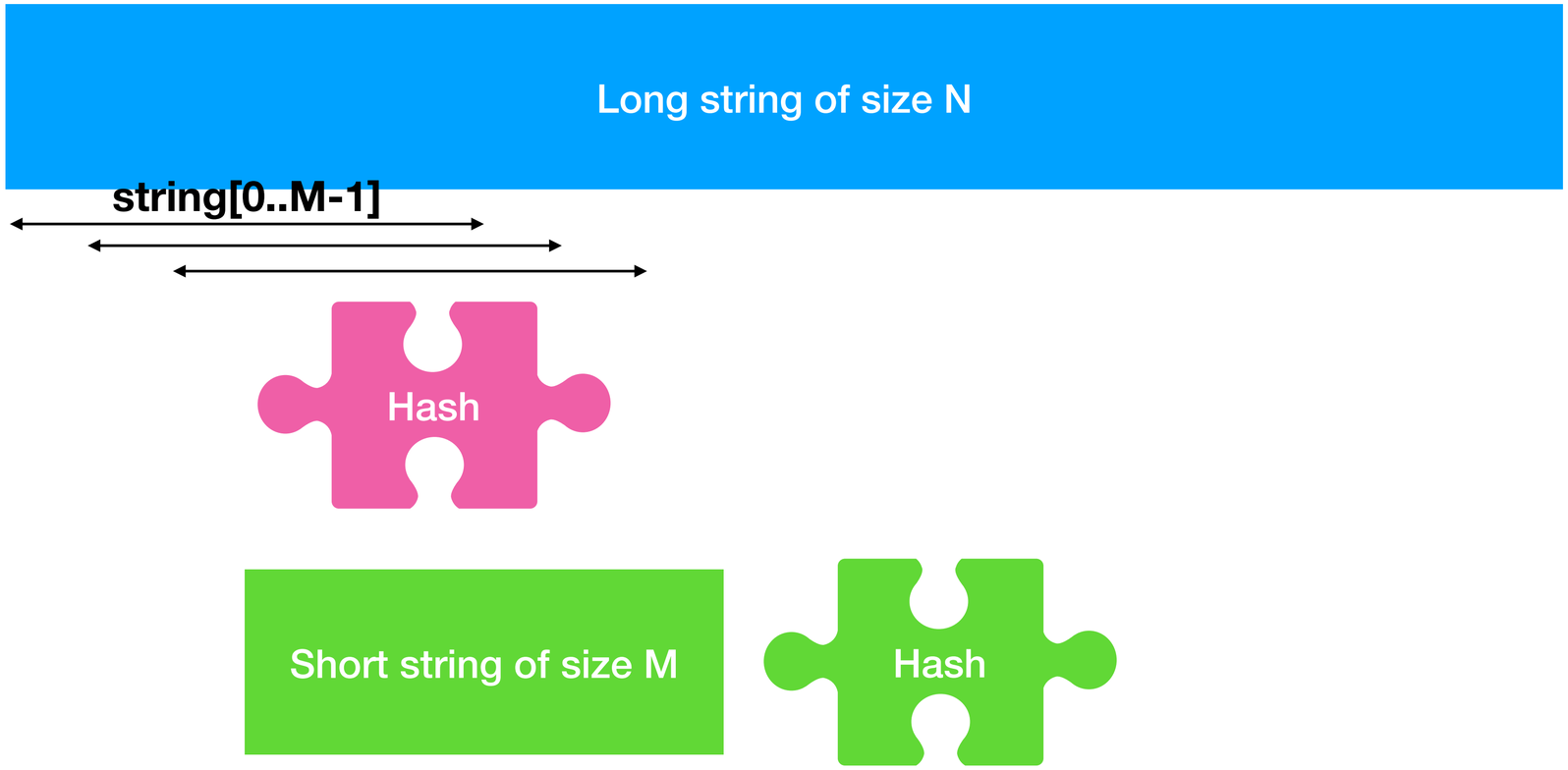




Rabin Karp


Long string of size N
Present, if yes, where ?
Short string of size M

Rabin Karp


Long string of size N
string[0..M-1]
Hash
Short string of size M
Hash